



Green University of Bangladesh

*Department of Computer Science and Engineering (CSE)
Semester: (Spring, Year: 2026), B.Sc. in CSE (Day)*

Secure Password Generator & Saver

Course Title: Web Programming

Course Code: CSE 301

Section: 241_D1

Students Details

Name	ID
Julqarnaeen Ahmed Jihad	241002046
Ibnul Hossain	241002023

Submission Date: 28 April, 2026

Course Teacher's Name: Most. Rokeya Khatun

Assignment Status

Marks:

Signature:

Comments:

Date:

Contents

1	Introduction	3
1.1	Overview	3
1.2	Motivation	3
1.3	Problem Definition	3
1.3.1	Problem Statement	3
1.3.2	Complex Engineering Problem (CEP) Attributes	4
1.4	Design Goals/Objectives	4
1.5	Application	5
2	Design/Development/Implementation of the Project	6
2.1	Introduction	6
2.2	Technologies Used	6
2.3	Project File Structure	6
2.4	Database Design	7
2.4.1	SQL Setup Script	7
2.5	Workflow Summary	7
2.6	All PHP Source Code	7
2.6.1	db_connect.php	7
2.6.2	index.php	8
2.6.3	save_password.php	14
2.6.4	delete_password.php	14
3	Performance Evaluation	15
3.1	Simulation Environment	15
3.2	Testing Strategy	15
3.3	Results Analysis	15
3.3.1	Test 1: Generate Password	15

3.3.2	Test 2: Strength Indicator	16
3.3.3	Test 3: Save Password	17
3.3.4	Test 4: Search Password	18
3.3.5	Test 5: Delete Password	18
3.4	Result Discussion	18
4	Conclusion	19
4.1	Discussion	19
4.2	Limitations	19
4.3	Scope of Future Work	19
	References	20

Chapter 1

Introduction

1.1 Overview

This project is a web-based **Password Generator and Saver** system. It is developed using PHP, JavaScript, HTML/CSS, and MySQL (MariaDB through XAMPP). The system can generate random passwords based on selected character rules, evaluate password strength, and save credentials (account name, username/email, and password) into a local database table.

1.2 Motivation

Many users still create weak passwords and store them manually in insecure places. This project was developed to solve three common problems in a simple local web application:

- Generate strong random passwords quickly.
- Save credentials for future use.
- Search and delete saved credentials from one interface.

1.3 Problem Definition

1.3.1 Problem Statement

A secure digital workflow needs strong and unique passwords for different accounts. Manual password creation often leads to short, repeated, or easy-to-guess passwords. Also, unmanaged password storage creates usability and security issues. This project addresses these issues by combining password generation, storage, search, and deletion features in one lightweight system.

1.3.2 Complex Engineering Problem (CEP) Attributes

Table 1.1 summarizes how the project maps to CEP attributes.

Table 1.1: CEP Attribute Mapping for This Project

P Attribute	How Addressed in This Project
P1: Depth of knowledge required	Requires web programming knowledge (PHP, JavaScript, SQL) and client-server interaction design.
P2: Range of conflicting requirements	Balances simplicity of UI with multiple functions (generate, save, search, delete).
P3: Depth of analysis required	Needs analysis of data flow from form input to database operations and output rendering.
P4: Familiarity of issues	Includes common web issues: input handling, SQL query design, responsive layout, and CRUD logic.
P5: Extent of applicable codes	Uses PHP (backend), SQL (data persistence), JavaScript (generator logic), and HTML/CSS (frontend).
P6: Extent of stakeholder involvement	Intended for end users who need password management and developers maintaining local systems.
P7: Interdependence	UI, JavaScript generator, PHP scripts, and database table are tightly connected for full functionality.

1.4 Design Goals/Objectives

The major objectives of this project are:

- Build a user-friendly password generation interface.
- Allow user-defined password length and character options.
- Display password strength (Weak/Medium/Strong).
- Save generated or custom passwords in MySQL database.
- Show all saved passwords in a table view.
- Add search functionality by account name.
- Add delete functionality for saved records.

1.5 Application

This project can be used in:

- Student web programming lab demonstrations.
- Small personal password management (local environment).
- CRUD and full-stack PHP practice tasks.
- Introductory security awareness sessions (password quality concept).

Chapter 2

Design/Development/Implementation of the Project

2.1 Introduction

This chapter explains the complete design and implementation of the Password Generator and Saver project based on the actual project files.

2.2 Technologies Used

- **Frontend:** HTML, CSS, JavaScript
- **Backend:** PHP
- **Database:** MariaDB/MySQL (XAMPP)
- **Web Server:** Apache (XAMPP)
- **Database Tool:** phpMyAdmin
- **Operating System:** Windows

2.3 Project File Structure

- `index.php` — main user interface, password generator, list and search view
- `db_connect.php` — database connection and table initialization
- `save_password.php` — saves submitted credentials
- `delete_password.php` — deletes selected credential by ID

2.4 Database Design

The project uses one table named vault:

- id (INT, AUTO_INCREMENT, PRIMARY KEY)
- account_name (VARCHAR(100))
- username (VARCHAR(150))
- password_val (VARCHAR(255))

2.4.1 SQL Setup Script

```
1 CREATE DATABASE IF NOT EXISTS password_db;  
2 USE password_db;  
3  
4 CREATE TABLE IF NOT EXISTS vault (  
5     id INT AUTO_INCREMENT PRIMARY KEY,  
6     account_name VARCHAR(100),  
7     username VARCHAR(150),  
8     password_val VARCHAR(255)  
9 );
```

Listing 2.1: Database and Table Setup Script

2.5 Workflow Summary

- User selects password options and clicks Generate.
- JavaScript creates random password and shows strength.
- Generated password is auto-copied into save form password field.
- User submits save form.
- save_password.php inserts data into vault table.
- Main page shows saved passwords and supports search.
- User can delete any row using delete_password.php.

2.6 All PHP Source Code

2.6.1 db_connect.php

```

1 <?php
2 /*
3 SQL SETUP (run this in phpMyAdmin first)
4
5 CREATE DATABASE IF NOT EXISTS password_db;
6 USE password_db;
7
8 CREATE TABLE IF NOT EXISTS vault (
9     id INT AUTO_INCREMENT PRIMARY KEY,
10     account_name VARCHAR(100),
11     username VARCHAR(150),
12     password_val VARCHAR(255)
13 );
14 */
15
16 // This connects to the database
17 $servername = "localhost";
18 $username = "root";
19 $db_pass = "";
20 $dbname = "password_db";
21
22 $conn = mysqli_connect($servername, $username, $db_pass);
23
24 if (!$conn) {
25     die("Connection failed: " . mysqli_connect_error());
26 }
27
28 // Create database if not exists
29 $sql_db = "CREATE DATABASE IF NOT EXISTS $dbname";
30 mysqli_query($conn, $sql_db);
31
32 // Select database
33 mysqli_select_db($conn, $dbname);
34
35 // Create table if not exists
36 $sql_table = "CREATE TABLE IF NOT EXISTS vault (
37     id INT AUTO_INCREMENT PRIMARY KEY,
38     account_name VARCHAR(100),
39     username VARCHAR(150),
40     password_val VARCHAR(255)
41 )";
42 mysqli_query($conn, $sql_table);
43 ?>

```

Listing 2.2: db_connect.php

2.6.2 index.php

```

1 <?php
2 include "db_connect.php";
3
4 $search = "";
5 if (isset($_GET['search'])) {
6     $search = $_GET['search'];
7 }
8 ?>

```

```

9 <!DOCTYPE html>
10 <html>
11 <head>
12     <meta charset="UTF-8">
13     <meta name="viewport" content="width=device-width, initial-
14     scale=1.0">
15     <title>Password Generator and Saver</title>
16     <style>
17         body {
18             font-family: Arial, sans-serif;
19             background: #f4f6f8;
20             margin: 0;
21             padding: 20px;
22         }
23         h1 {
24             margin-top: 0;
25             color: #333;
26         }
27         .container {
28             display: grid;
29             grid-template-columns: 1fr 1fr;
30             gap: 20px;
31         }
32         .card {
33             background: #fff;
34             border: 1px solid #d5dbe1;
35             padding: 15px;
36             border-radius: 4px;
37         }
38         label {
39             display: block;
40             margin-top: 10px;
41             margin-bottom: 5px;
42             color: #333;
43         }
44         input[type="text"],
45         input[type="email"],
46         input[type="number"] {
47             width: 100%;
48             padding: 10px;
49             border: 1px solid #bfc7d1;
50             border-radius: 3px;
51             box-sizing: border-box;
52         }
53         .check-group {
54             margin-top: 10px;
55         }
56         .check-group label {
57             display: inline-block;
58             margin-right: 10px;
59             margin-top: 0;

```

```

66     }
67
68     button,
69     .btn-link {
70         margin-top: 12px;
71         background: #2f6fb2;
72         color: white;
73         border: 1px solid #2a639f;
74         padding: 10px 14px;
75         border-radius: 3px;
76         cursor: pointer;
77         text-decoration: none;
78         display: inline-block;
79     }
80
81     .btn-link.delete {
82         background: #7a7f87;
83         border-color: #676b72;
84         padding: 6px 10px;
85         margin-top: 0;
86     }
87
88     .strength {
89         margin-top: 10px;
90         font-weight: bold;
91     }
92
93     table {
94         width: 100%;
95         border-collapse: collapse;
96         margin-top: 10px;
97     }
98
99     table th,
100    table td {
101        border: 1px solid #cfd6dd;
102        padding: 10px;
103        text-align: left;
104    }
105
106    table th {
107        background: #e9eef3;
108    }
109
110    .search-row {
111        display: flex;
112        gap: 10px;
113        align-items: center;
114        margin-top: 10px;
115    }
116
117    .search-row input {
118        flex: 1;
119    }
120
121    @media (max-width: 850px) {
122        .container {
123            grid-template-columns: 1fr;

```

```

124     }
125 }
126 </style>
127 </head>
128 <body>
129     <h1>Password Generator and Saver</h1>
130
131     <div class="container">
132         <div class="card">
133             <h3>1) Generate Password</h3>
134
135             <label>Password Length</label>
136             <input type="number" id="p_length" value="10" min="4"
max="50">
137
138             <div class="check-group">
139                 <label><input type="checkbox" id="upper" checked>
Uppercase</label>
140                 <label><input type="checkbox" id="lower" checked>
Lowercase</label>
141                 <label><input type="checkbox" id="number" checked>
Numbers</label>
142                 <label><input type="checkbox" id="symbol"> Symbols
</label>
143             </div>
144
145             <button type="button" onclick="generatePassword()">
Generate</button>
146
147             <label>Generated Password</label>
148             <input type="text" id="generated_password" readonly>
149
150             <div class="strength" id="strength_text">Strength: -</
div>
151         </div>
152
153         <div class="card">
154             <h3>2) Save Password</h3>
155             <form action="save_password.php" method="POST">
156                 <label>Account Name</label>
157                 <input type="text" name="account_name" placeholder
="Example: Facebook" required>
158
159                 <label>Username / Email</label>
160                 <input type="text" name="username" placeholder="
Example: mymail@gmail.com" required>
161
162                 <label>Password</label>
163                 <input type="text" name="password_val" id="
save_password_input" required>
164
165                 <button type="submit" name="submit">Save</button>
166             </form>
167         </div>
168     </div>
169
170     <div class="card" style="margin-top: 20px;">
171         <h3>3) Saved Passwords</h3>

```

```

172     <form action="index.php" method="GET" class="search-row">
173         <input type="text" name="search" placeholder="Search
174 by account name" value="<?php echo htmlspecialchars($search);
?>">
175         <button type="submit">Search</button>
176         <a href="index.php" class="btn-link">Clear</a>
177     </form>
178
179     <table>
180         <tr>
181             <th>ID</th>
182             <th>Account Name</th>
183             <th>Username/Email</th>
184             <th>Password</th>
185             <th>Action</th>
186         </tr>
187
188         <?php
189         // Get data with optional search
190         if ($search != "") {
191             $sql = "SELECT * FROM vault WHERE account_name
LIKE '%$search%' ORDER BY id DESC";
192         } else {
193             $sql = "SELECT * FROM vault ORDER BY id DESC";
194         }
195
196         $result = mysqli_query($conn, $sql);
197
198         if ($result && mysqli_num_rows($result) > 0) {
199             while ($row = mysqli_fetch_assoc($result)) {
200                 echo "<tr>";
201                 echo "<td>" . $row['id'] . "</td>";
202                 echo "<td>" . htmlspecialchars($row['
account_name']) . "</td>";
203                 echo "<td>" . htmlspecialchars($row['username'
]) . "</td>";
204                 echo "<td>" . htmlspecialchars($row['
password_val']) . "</td>";
205                 echo "<td><a class='btn-link delete' href='
delete_password.php?id=" . $row['id'] . "' onclick='return
confirm(\"Delete this entry?\")'>Delete</a></td>";
206                 echo "</tr>";
207             }
208         } else {
209             echo "<tr><td colspan='5'>No saved passwords found
.</td></tr>";
210         }
211         ?>
212     </table>
213 </div>
214
215 <script>
216     // This generates password based on selected options
217     function generatePassword() {
218         var p_length = parseInt(document.getElementById('
p_length').value);
219         var useUpper = document.getElementById('upper').

```

```

checked;
220     var useLower = document.getElementById('lower').
checked;
221     var useNumber = document.getElementById('number').
checked;
222     var useSymbol = document.getElementById('symbol').
checked;

223
224     var upperChars = 'ABCDEFGHIJKLMNOPQRSTUVWXYZ';
225     var lowerChars = 'abcdefghijklmnopqrstuvwxyz';
226     var numberChars = '0123456789';
227     var symbolChars = '!@#$%^&*()_+{}[]';
228
229     var allChars = '';
230
231     if (useUpper) {
232         allChars += upperChars;
233     }
234     if (useLower) {
235         allChars += lowerChars;
236     }
237     if (useNumber) {
238         allChars += numberChars;
239     }
240     if (useSymbol) {
241         allChars += symbolChars;
242     }
243
244     if (allChars === '') {
245         alert('Please select at least one character type.'
);
246         return;
247     }
248
249     var newPassword = '';
250     for (var i = 0; i < p_length; i++) {
251         var randomIndex = Math.floor(Math.random() *
allChars.length);
252         newPassword += allChars[randomIndex];
253     }
254
255     document.getElementById('generated_password').value =
newPassword;
256     document.getElementById('save_password_input').value =
newPassword;
257
258     // Simple strength rules from requirement
259     var strengthText = 'Strength: Weak';
260     var strengthColor = 'red';
261
262     if (p_length >= 12 && useSymbol) {
263         strengthText = 'Strength: Strong';
264         strengthColor = 'green';
265     } else if (p_length >= 8 && useNumber) {
266         strengthText = 'Strength: Medium';
267         strengthColor = 'orange';
268     }
269

```

```

270         var strengthEl = document.getElementById('
strength_text');
271         strengthEl.textContent = strengthText;
272         strengthEl.style.color = strengthColor;
273     }
274     </script>
275 </body>
276 </html>

```

Listing 2.3: index.php

2.6.3 save_password.php

```

1 <?php
2 include "db_connect.php";
3
4 // Save form data into database
5 if (isset($_POST['submit'])) {
6     $account_name = $_POST['account_name'];
7     $username = $_POST['username'];
8     $password_val = $_POST['password_val'];
9
10    $sql = "INSERT INTO vault (account_name, username,
password_val) VALUES ('$account_name', '$username', '
$password_val')";
11
12    mysqli_query($conn, $sql);
13 }
14
15 // Go back to main page
16 header("Location: index.php");
17 exit();
18 ?>

```

Listing 2.4: save_password.php

2.6.4 delete_password.php

```

1 <?php
2 include "db_connect.php";
3
4 // Delete by id from URL
5 if (isset($_GET['id'])) {
6     $id = $_GET['id'];
7     $sql = "DELETE FROM vault WHERE id = '$id'";
8     mysqli_query($conn, $sql);
9 }
10
11 // Go back to main page
12 header("Location: index.php");
13 exit();
14 ?>

```

Listing 2.5: delete_password.php

Chapter 3

Performance Evaluation

3.1 Simulation Environment

The project was tested in a local development environment:

- XAMPP (Apache + MariaDB)
- PHP runtime from XAMPP
- phpMyAdmin for database inspection
- Windows operating system

3.2 Testing Strategy

Testing was done using functional test scenarios:

- Password generation with different length and character options.
- Password strength label changes (Weak/Medium/Strong).
- Save operation and table insertion in vault.
- Search by account name.
- Delete row from saved list.
- Responsive UI check on smaller screen width.

3.3 Results Analysis

3.3.1 Test 1: Generate Password

Expected: New password generated according to selected rules.

Password Generator and Saver

1) Generate Password

Password Length

☒ Uppercase
 ☒ Lowercase
 ☒ Numbers
 ☐ Symbols

Generate

Generated Password

Strength: -

2) Save Password

Account Name

Username / Email

Password

Save

3) Saved Passwords

Search by account name

Search **Clear**

ID	Account Name	Username/Email	Password	Action
1	Facebook	facebook@gmail.com	5M%57q]sz]3]	Delete

Figure 3.1: Main Interface of Password Generator and Saver

Password Generator and Saver

1) Generate Password

Password Length

☒ Uppercase
 ☒ Lowercase
 ☒ Numbers
 ☒ Symbols

Generate

Generated Password

Strength: Strong

Figure 3.2: Generated Password Output

3.3.2 Test 2: Strength Indicator

Expected: Weak, Medium, and Strong labels update correctly based on rule logic.

Password Generator and Saver

1) Generate Password

Password Length

☒ Uppercase ☒ Lowercase ☒ Numbers ☐ Symbols

Generate

Generated Password

Strength: Medium

Figure 3.3: Password Strength Indicator Result

3.3.3 Test 3: Save Password

Expected: Record is inserted into database and displayed in saved password table.

2) Save Password

Account Name

Username / Email

Password

Save

Figure 3.4: Save Form with Input Data

3) Saved Passwords

Search by account name

ID	Account Name	Username/Email	Password	Action
2	messenger	messenger@gmail.com	RP5i3njIH0oM	Delete
1	Facebook	facebook@gmail.com	5M%57q)sz]3]	Delete

Figure 3.5: Saved Passwords Table After Insert

3.3.4 Test 4: Search Password

Expected: Filtered rows are shown based on account name keyword.

3) Saved Passwords

messenger

ID	Account Name	Username/Email	Password	Action
2	messenger	messenger@gmail.com	RP5i3njIH0oM	Delete

Figure 3.6: Search Result by Account Name

3.3.5 Test 5: Delete Password

Expected: Selected entry is deleted and removed from table after redirect.

3) Saved Passwords

Search by account name

ID	Account Name	Username/Email	Password	Action
1	Facebook	facebook@gmail.com	5M%57q)sz]3]	Delete

Figure 3.7: Delete Confirmation Popup

3.4 Result Discussion

The project successfully demonstrates end-to-end integration of frontend interaction, backend PHP processing, and database persistence. All major functional requirements were implemented and validated in local testing.

	id	account_name	username	password_val
☐ Edit Copy Delete	1	Facebook	facebook@gmail.com	5M%57q)sz]3]

Figure 3.8: Database Table View in phpMyAdmin

Chapter 4

Conclusion

4.1 Discussion

The Password Generator and Saver project achieved its core goals. The application supports configurable password generation, simple password strength feedback, credential saving, search, and deletion. It is suitable as a compact academic project for demonstrating practical PHP and MySQL concepts.

4.2 Limitations

Current limitations are:

- Passwords are stored in plain text in the database.
- SQL queries are directly concatenated (risk of SQL injection).
- No user authentication and no access control.
- No encryption of saved credentials.

4.3 Scope of Future Work

Future improvements can include:

- Use prepared statements for database security.
- Encrypt password values before storing in database.
- Add login and session-based user management.
- Add copy-to-clipboard and show/hide password features.
- Add password update/edit functionality.

References

- [1] Ramez Elmasri and Shamkant B. Navathe. *Fundamentals of Database Systems*. Pearson, Boston, 7th edition, 2016. Available at: <https://www.pearson.com/en-us/subject-catalog/p/fundamentals-of-database-systems/P200000003546>
- [2] Thomas M. Connolly and Carolyn E. Begg. *Database Systems: A Practical Approach to Design, Implementation, and Management*. Pearson, Harlow, England, 6th edition, 2015. Available at: <https://www.pearson.com/en-gb/subject-catalog/p/database-systems-a-practical-approach-to-design-implementation-and-management/P200000004519>
- [3] PHP Documentation Team. *PHP Manual: MySQLi Extension*. The PHP Group. Available at: <https://www.php.net/manual/en/book.mysqli.php>
- [4] OWASP Foundation. *Password Storage Cheat Sheet*. OWASP Cheat Sheet Series. Available at: <https://cheatsheetseries.owasp.org/cheatsheets/PasswordStorageCheatSheet.html>